

# TP2

## Utilisation du langage assembleur sur ARM Cortex M0+

Dubresson Matteo

– 26 septembre 2026 –

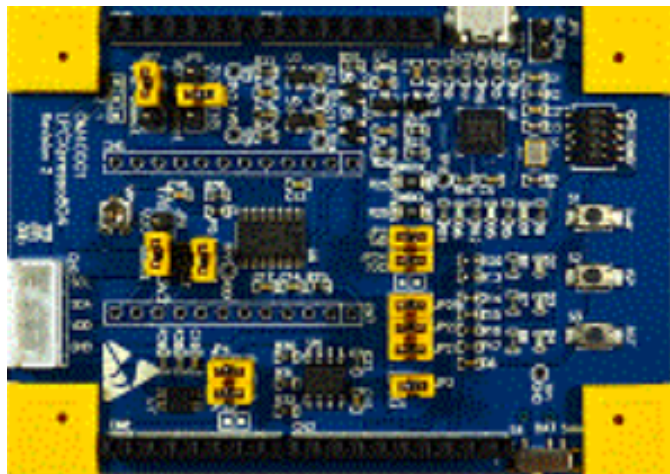


FIGURE 1 – LPC804 utilisé

## Table des matières

|   |                |   |
|---|----------------|---|
| 1 | Préparation    | 2 |
| 2 | Manipulation 1 | 3 |
| 3 | Manipulation 2 | 3 |
| 4 | Manipulation 3 | 4 |

# 1 Préparation

## P.1

Dans la documentation du LPC804, on peut retrouver les adresses de ces différents registres.

SYSAHBCLKCTRL0 :  $0x4004\ 8080$  |  $0100\ 0000\ 0000\ 0000\ 0100\ 1000\ 0000\ 1000\ 0000$

DIR0 :  $0xA000\ 2000$  |  $1010\ 0000\ 0000\ 0000\ 0010\ 0000\ 0000\ 0000$

## P.2

Dans la documentation des processeurs ARMv6-M, on retrouve page A1-26 que les instructions sont codées sur 32 bits.

## P.3

La ligne 10 du code active l'horloge des périphériques GPIO et IOCON. On réalise un |= pour ne pas changer les périphériques déjà activés. On combine également les deux périphériques avec un | pour pouvoir réaliser l'opération en une seule fois.

## P.4

Dans le code assembleur présenté, on retrouve plusieurs instructions :

- `movs r3,#x` : permet de mettre une valeur x dans un registre r3.
- `lsls r2,r3,#y` : permet de décaler les bits du registre r3 vers la gauche de y positions et la stocker dans r2.
- `ldr r1,[r0,r2]` : permet de lire la valeur à l'adresse contenue dans r0 + r2 et de la stocker dans r1.
- `orrs r1,r3` : fait `r1 |= r3`.
- `str r1,[r0,r2]` : permet d'écrire la valeur contenue dans r1 à l'adresse contenue dans r0 + r2.

## P.5

Le # sert à désigner une valeur immédiate et non un registre.

Le [] sert à désigner une adresse mémoire.

## P.6

On commence par mettre  $0xA0$  dans *R3*, on le décale de 24 bits puis on le met dans *R2*. On réalise la même opération avec  $0x80$  mais on le décale de 6 bits. En sommant les deux dans *R3*, on obtient  $0xA000\ 2000$  qui est l'adresse du registre DIR0.

On peut donc récupérer la valeur du registre DIR0, la suite du code servant à d'autres opérations.

## 2 Manipulation 1

### 1.1

Parmi les différents registres utilisés :

- SYSAHBCLKCTRL0 : permet d'activer l'horloge des périphériques GPIO et IOCON.
- DIR0 : permet de configurer les broches du port 0 en entrée ou en sortie.
- GPIO\_PORT\_NOT0 : permet d'inverser l'état du port 0.

### 1.2

Lorsqu'on implémente le code dans le microcontrôleur, on peut observer que la LED s'allume et s'éteint à une période de 0,57 s

### 1.3

Lorsqu'on lance le code dans l'onglet désassembleur, on constate que 48 instructions sont nécessaires pour exécuter la fonction main.

## 3 Manipulation 2

### 2.1

Les directives qui ne génèrent pas de code sont :

- .syntax
- .equ
- boucle / bne boucle

Tandis que les instructions qui génèrent du code sont :

- ldr
- str
- subs

### 2.2

Les instructions ont pour tâche

- ldr R,#x : permet de charger la valeur x dans le registre R.
- str R1,[R2,#0] : permet d'écrire la valeur contenue dans R1 à l'adresse contenue dans R2.
- subs R6,#1 : fait R6-=1

On peut réécrire l'algorithme sous forme de pseudo-code :

```

    tant que
        GPIO_PORT_NOT0 = 0xa0002300
        LED4 = (1 << 11)
        init_cpt = 2000
        stocker LED4 à l'adresse GPIO_PORT_NOT0
    tant que init_cpt non nul
        init_cpt - = 1

```

## 2.3

Lors de l'exécution des instructions 0x334 à 0x338, on fait :

- 334. `ldr r6 [pc,#20] @0x34c < boucle + 22 >` : on charge dans r6 la valeur contenue à l'adresse `pc+20`, soit 0x34c. On obtient donc `r6 = 2000`.
- 336. `subs r6,#1` : on décrémente r6 de 1
- 338. `bne.n 0x336 < main + 54 >` : on teste si r6 est nul, si ce n'est pas le cas on retourne à l'adresse 0x336.

## 2.4

Maintenant qu'on a optimisé le code, on peut observer que la LED s'allume et s'éteint avec une période beaucoup plus courte, d'environ 0,15 s (compliquer à mesurer précisément car clignote très vite).

## 2.5

Si on analyse l'implémentation des instructions `ldr`, on peut constater que

## 2.6

Dans la documentation, on comprend que le `s` sert à faire une mise à jour du drapeau de condition.

## 2.7

`Bne` signifie *branch if not equal*, ainsi dans une instruction du type `bne boucle` :

- si le drapeau `Z == 0` : on continue la boucle
- si le drapeau `Z == 1` : on sort de la boucle

# 4 Manipulation 3

## 3.1

On a intégré la boucle infinie au code assembleur dans la fonction `main2`. En faisant ça le code rédigé à la main est beaucoup plus optimisé : la boucle d'attente ne fait plus que deux instructions

(subs, bne), ce qui veut dire qu'elle s'exécutera plus vite qu'en C, donc on aura un gain de temps significatif.

## 3.2

Dans le programme en C on déclare le `main2()` à la ligne 14, et comme dans le code assembleur on fait un global `.main2`, on rend visible la fonction pour assurer sa liaison entre les différents fichiers.

## 3.3 / 3.4

Comme le code qu'on rajoute dans le fichier assembleur correspond à l'initialisation de la tâche qu'on fait, et non les consignes qu'on effectue en boucle, le gain de temps sera non significatif.

## 3.5

Le programme en C prend une quinzaine de lignes à écrire, et est moins complexe pour un utilisateur à faire. Le programme assembleur prend 20 lignes de plus à écrire, et est moins évident à écrire car on doit détailler toutes les opérations et faire bien attention aux registres qu'on utilise à chaque fois.

## 3.6

Cependant, malgré les réponses données en 3.5, étant donné qu'on peut agir directement sur les consignes qu'on demande, on peut simplifier grandement le temps de calcul. Le code assembleur a donc cette utilité : éviter de se reposer sur des instructions pré-construites par le langage C qui ne sont pas optimisées dans tout les cas de figure.